American University of Sharjah

Department of Computer Science and Engineering

CMP 310: Operating Systems

Professor: Dr. Shadi Banitaan

Mini Project Submission

Group 9 Members:

Karim Rashad – b00079266

Khalifa Altheeb – b00095261

Submission Date

May 2nd, 2026

1. Introduction

The project develops a multithreaded Producer-Consumer system through C programming with POSIX pthreads and semaphore and mutex protection. The program operates a shared circular buffer with fixed capacity which producer threads use to add randomly generated items and consumer threads use to take away and handle those items.

The project's main objective is to achieve proper synchronization between multiple producer threads and consumer threads. The program uses semaphores and mutual exclusion to protect against race conditions and buffer overflow and buffer underflow.

2. Design Decisions

The shared buffer was implemented as a circular bounded buffer. The head and tail positions use circular indexing to wrap around at the buffer boundaries. This method enables the fixed-size buffer to operate at its maximum capacity.

The system used two semaphores to manage its operations:
- empty_slots: tracks the number of empty spaces available in the buffer.
- filled_slots: tracks the number of items currently available for consumers.

A mutex was used to protect critical sections where the shared buffer is accessed or modified. The system ensures that only one thread can handle item insertion or removal operations.

Each producer generates 20 real items. The command-line arguments determine the number of producers, the number of consumers, and the buffer size.

The system used Poison Pill as its termination method which allows processes to stop running gracefully. The main thread adds one poison pill for each consumer after all producers have completed their tasks. The consumer process exits safely when it receives a poison pill.

3. Synchronization Logic

The producer first waits on empty_slots. The producer blocks when the buffer reaches its full capacity until the system frees up storage space. The process begins with mutex acquisition followed by buffer item insertion and ending with mutex release while filled_slots post notification to consumers.

The consumer first waits on filled_slots. The consumer blocks until an item becomes available when the buffer reaches empty status. The process starts with mutex acquisition which allows item removal from the buffer and ends with mutex release while empty_slots post notification to producers about available space.

The system prevents busy waiting because semaphores block threads instead of allowing them to continuously check buffer conditions.

4. Bonus Features

The implementation of priority handling used two circular queues which included one queue for urgent items and another queue for standard items. The system assigns each item a priority value which uses 0 to represent normal priority and 1 to represent urgent priority. Consumers always remove urgent items first if urgent items are available. Each priority queue continues to maintain its original FIFO structure.

urgent items make up 25 percent of the items that get produced. The team established an urgent marking system which marked every fourth item as urgent.

The system recorded both throughput and latency metrics. The program records the enqueue timestamp when an item is inserted and the dequeue timestamp when an item is removed. The system calculates latency using the following equation:

latency = dequeue time - enqueue time

The system calculates throughput using the following equation:

throughput = total consumed real items / total execution time

5. Bonus Results

Run	Producers	Consumers	Buffer Size	Average Latency	Throughput
Small Buffer	8	8	2	0.000121 sec	7376.19 items/sec
Large Buffer	8	8	32	0.000303 sec	14225.87 items/sec

The larger buffer achieved higher throughput because producers and consumers blocked less often. The small buffer caused more synchronization blocking because only two items could fit in the buffer at once.

```
@KhalifaAltheeb ➜/workspaces/CMP310_Mini_Project (main) $ ./producer_consumer 8 8 32
[Consumer-8] Finished consuming.
[Consumer-1] Received POISON_PILL. Exiting.
[Consumer-1] Finished consuming.

========== Final Statistics ==========
Total real items produced: 160
Total real items consumed: 160
Urgent items consumed: 40
Normal items consumed: 120
Average latency: 0.000303 seconds
Throughput: 14225.87 items/second
Total execution time: 0.011247 seconds
All consumers exited successfully.
@KhalifaAltheeb ➜/workspaces/CMP310_Mini_Project (main) $
```

```
@KhalifaAltheeb ➜/workspaces/CMP310_Mini_Project (main) $ ./producer_consumer 8 8 2
[Consumer-2] Finished consuming.
[Consumer-7] Received POISON_PILL. Exiting.
[Consumer-7] Finished consuming.

========== Final Statistics ==========
Total real items produced: 160
Total real items consumed: 160
Urgent items consumed: 40
Normal items consumed: 120
Average latency: 0.000121 seconds
Throughput: 7376.19 items/second
Total execution time: 0.021691 seconds
All consumers exited successfully.
@KhalifaAltheeb ➜/workspaces/CMP310_Mini_Project (main) $
```

6. Challenges

The researchers needed to prevent race conditions which occurred when multiple producers and consumers used the buffer simultaneously. The implementation of a mutex system which controlled all buffer operations resolved this issue.

The team faced difficulties with the process of terminating consumer processes. The consumers will remain active until all producers complete their work because they will be waiting for a buffer that currently holds no items. The main thread creates one poison pill for each consumer after it joins all producer threads.

The team needed to develop a system which would handle priority requests while still maintaining first-in-first-out execution order. The problem required a solution which used two distinct circular queues: an urgent queue and a normal queue.

7. Individual Contributions

- Khalifa: Implemented producer and consumer thread logic, and Prepared README, report, testing, and demo video.
- Karim: Implemented semaphores, mutex, and circular buffer synchronization, and Implemented bonus priority handling and metrics.

8. Conclusion

The project successfully implements a synchronized Producer-Consumer application using pthreads, semaphores, and mutexes. The program operates correctly with multiple producers and consumers while it handles race conditions and uses poison pills to achieve controlled shutdown of its operations. The bonus features also provide priority-based consumption and performance metrics.